REKURSIONEN

Coders.Eleven - JAVA

#1 Allgemein

Beispiel

Klassifikation

#2 Rekursionsarten

Lineare Rekursion

Verzweigte Rekursion

Wechselseitige Rekursion

Verschachtelte Rekursion

#3 Rekursion vs. Iteration

Vergleich



#1 Allgemein

Beispiel

Klassifikation



Beispiel

- Berechnung der Summe der Zahlen von 1 bis n
- Iterativ ist die Summe definiert durch

$$sum(n) = 1 + 2 + \ldots + n$$

- Rekursiv ist die Summe definiert durch

$$sum(n) = \begin{cases} 1 & , \text{falls } n = 1 \\ sum(n-1) + n & , \text{sonst} \end{cases}$$

Basisfall

Rekursionsschritt

Rekursionsschritt

- Selbstaufruf (Zerlegung in ein kleineres Problem)
  - Jeder Aufruf muss in irgendeiner Form einen Beitrag zum Erreichen der Abbruchbedingung leisten (Fortschritt)
- Problem lösen
  - Ergebnis des Selbstaufrufs verwenden (in Operationen)
  - Selbstaufruf entsprechend parametrisieren usw.

$$\texttt{sum(n)} = \begin{cases} \texttt{1} & \texttt{, falls n = 1} \\ \texttt{sum(n - 1) + n} & \texttt{, sonst} \end{cases}$$

Basisfall

- Beschreibt triviales Problem
  - Kann sofort „gelöst“ werden
    - Bsp.: Rückgabe eines sinnvollen Wertes
  - Kein Selbstaufruf

$$\text{sum}(n) = \begin{cases} 1 & , \text{falls } n = 1 \\ \text{sum}(n-1) + n & , \text{sonst} \end{cases}$$

Klassifikation

- Klassifikation nach Anzahl und Anordnung der rekursiven Aufrufe in einem Rekursionsschritt
  - Lineare Rekursion
  - Verzweigte Rekursion
  - Verschachtelte Rekursion
  - Wechselseitige Rekursion

#2 Rekursionsarten

Lineare Rekursion

Verzweigte Rekursion

Wechselseitige Rekursion

Verschachtelte Rekursion



Lineare Rekursion

- In jedem Rekursionsschritt gibt es genau einen rekursiven Aufruf
- Beispiel: rekursive Summenberechnung (siehe Folie 4 unten)

```java
private int sumRecursive(int num) {
    if(num <= 1) {
        return 1;
    } else {
        return sumRecursive( num: num - 1) + num;
    }
}
```

```java
private int sumIterative(int num) {
    int sum = 1;
    for(int i = 2; i <= num; i++) {
        sum += i;
    }
    return sum;
}
```

```java
System.out.println(sumIterative( num: 10));
System.out.println(sumRecursive( num: 10));
System.out.println(sumIterative( num: 100));
System.out.println(sumRecursive( num: 100));
```

```
55
55
5050
5050
```

Rekursive Implementierung – Ablauf

- Ablauf der Rekursion für n = 4

sumRecursive(4) = sumRecursive(3) + 4

sumRecursive(3) = sumRecursive(2) + 3

sumRecursive(2) = sumRecursive(1) + 2

sumRecursive(1) = 1

sumRecursive(2) = 1 + 2 = 3

sumRecursive(3) = 3 + 3 = 6

sumRecursive(4) = 6 + 4 = 10

Aufrufpfad

`return`-Pfad

Berechnung erfolgt am `return`-Pfad

Position des rekursiven Aufrufes

```java
private void output(int n) {
    if(n < 1) {
        System.out.print("Numbers ");
    } else {
        System.out.print(n + " ");
        output( n: n - 1);
    }
}
```

Ausgabe am Aufrufpfad

```java
output( n: 5);
System.out.println();
output2( n: 5);
```

```java
private void output2(int n) {
    if(n < 1) {
        System.out.print("Numbers ");
    } else {
        output2( n: n - 1);
        System.out.print(n + " ");
    }
}
```

Ausgabe am return-Pfad

```
5 4 3 2 1 Numbers
Numbers 1 2 3 4 5 5
```

Rückgabe verwenden

```
private String squeeze(String s, char c) {
    if(s.isEmpty()) {
        return s;
    }
    if(s.charAt(0) == c) {
        return squeeze(s.substring( beginIndex: 1), c);
    }
    return s.charAt(0) + squeeze(s.substring( beginIndex: 1), c);
}
```

Entfernt rekursiv alle Vorkommen des Zeichens `c` aus dem `String` `s` und liefert einen neuen `String`

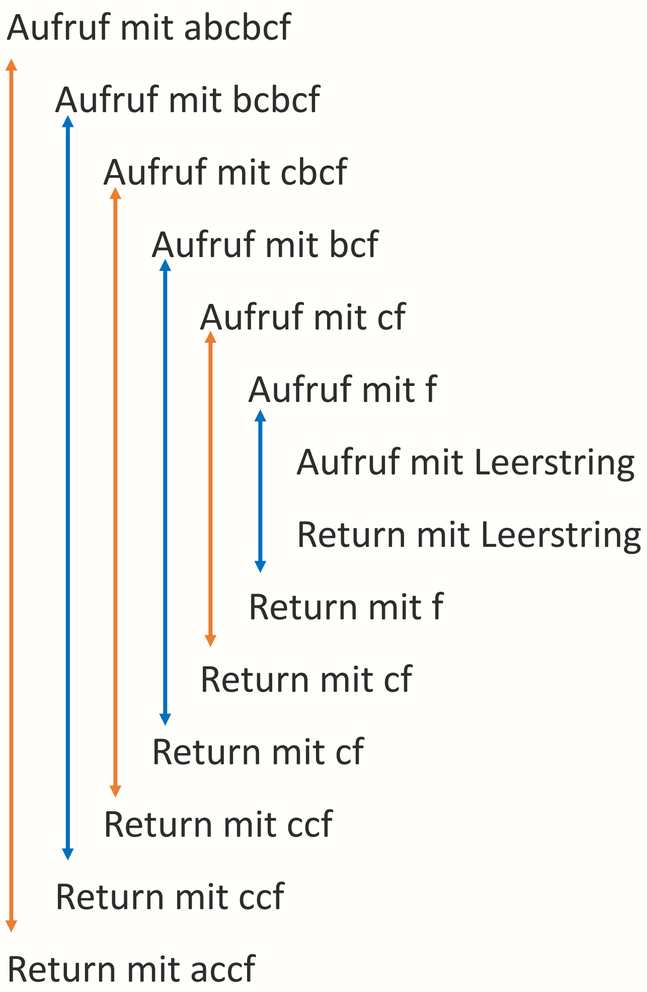
```
System.out.println(squeeze( s: "abcbcbdbf",  c: 'b'));
System.out.println(squeeze( s: "Hello World",  c: 'l'));
System.out.println(squeeze( s: "aaaa",  c: 'a'));
System.out.println(squeeze( s: "a b c d",  c: ' '));
```

```
accdf
Heo Word


abcd
```

Rekursive Implementierung – Ablauf

- Aufruf mit String abcbcf und Zeichen b




```java
private String squeeze(String s, char c) {
    if(s.isEmpty()) {
        return s;
    }
    if(s.charAt(0) == c) {
        return squeeze(s.substring( beginIndex: 1), c);
    }
    return s.charAt(0) + squeeze(s.substring( beginIndex: 1), c);
}
```

Endrekursion

- Jeder rekursive Aufruf ist die letzte Aktion der Methode
- Rekursiver Aufruf ersetzt damit den Originalaufruf mit angepassten Argumenten
- Beispiel: ggT-Berechnung

```
private int gcdRecursive(int a, int b) {
    if(a == b) {
        return a;
    } else if(a > b) {
        return gcdRecursive( a: a - b, b);
    } else {
        return gcdRecursive(a,  b: b - a);
    }
}
```

```
private int gcdIterative(int a, int b) {
    while(a != b) {
        if(a > b) {
            a -= b;
        } else {
            b -= a;
        }
    }
    return a;
}
```

```
System.out.println(gcdIterative( a: 24,  b: 16));
System.out.println(gcdRecursive( a: 24,  b: 16));
System.out.println(gcdIterative( a: 4399,  b: 166));
System.out.println(gcdRecursive( a: 4399,  b: 166));
```

```
8
8
83
83
```

Endrekursion mit zusätzlichen Parametern

- Rekursiver Aufruf in einer Berechnung ist nicht endrekursiv
  - Beispiel Summenberechnung
- Ein solcher Aufruf kann jedoch umgeschrieben werden
  - Zusätzliche überladene Methode
  - Zusätzliche Parameter bei dieser Methode
    - Akkumulierende Parameter

Endrekursion – Beispiel

- Summenberechnung mit 2 Methoden
  - Eine Methode mit ursprünglicher Signatur (nur ein Parameter)
  - Eine überladene Methode mit zusätzlichem Parameter

```java
private int sumRecursive2(int sum, int n) {
    if(n <= 1) {
        return sum;
    } else {
        return sumRecursive2( sum: sum + n,  n: n - 1);
    }
}
```

Endrekursive Form der Summenberechnung

```java
private int sumRecursive2(int num) {
    return sumRecursive2( sum: 1, num);
}
```

Methode mit ursprünglicher Signatur

Endrekursion – Ablauf

- Ablauf der Rekursion für `n = 4`:

```
sumRecursive2(4) = sumRecursive2(1, 4)
                 = sumRecursive2(5, 3)
                 = sumRecursive2(8, 2)
                 = sumRecursive2(10, 1)
                 = 10
```

Berechnung am Aufrufpfad – am `return`-Pfad wird nur das fertig berechnete Ergebnis zurückgegeben

Sinn der Endrekursion

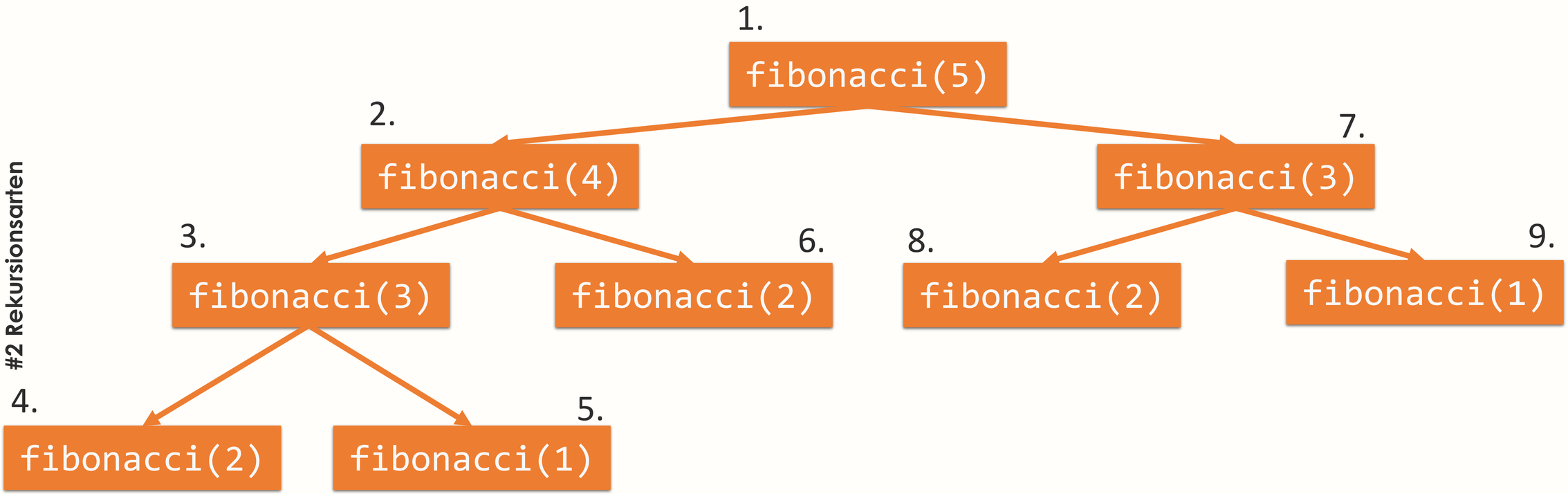
- Endrekursive Methoden lassen sich schnell in iterative überführen
- Manche Compiler erkennen Endrekursionen und entfernen diese automatisch
  - Daher rekursiv programmieren und die Umsetzung dem Compiler überlassen
    - Gilt allerdings nicht für Java
  - Iteration kann effizienter sein bzw. für größere Datenmengen funktionieren

Verzweigte Rekursion

- Jeder Aufruf, der nicht den Basisfall trifft, erzeugt zwei oder mehrere Aufrufe
  - Es entsteht ein Aufrufbaum
- Beispiel: Fibonacci-Zahlen

$$\texttt{fibonacci(n)} = \begin{cases} \texttt{1} & \texttt{, falls n = 1 oder n = 2} \\ \texttt{fibonacci(n - 1) + fibonacci(n - 2)} & \texttt{, sonst} \end{cases}$$

Aufrufbaum für `fibonacci(5)`



- Anzahl der Aufrufe wächst schnell an!

Wechselseitige Rekursion

- Der rekursive Aufruf erfolgt indirekt
  - Es können auch mehr als zwei Methoden an der Rekursion beteiligt sein
  - Wird auch als indirekte Rekursion oder alternierende Rekursion bezeichnet
- Beispiel
  - Methode a ruft Methode b auf
  - Methode b ruft Methode a auf
- Hinweis
  - Beim Lösen eines Problems P werden rekursive Lösungen für Subprobleme verwendet, die selbst nicht kleinere Instanzen von P sind

Wechselseitige Rekursion – Beispiel

```java
private boolean even(int n) {
    if(n == 0) {
        return true;
    }
    return odd( n: n - 1);
}
```

```java
private boolean odd(int n) {
    if(n == 0) {
        return false;
    }
    return even( n: n - 1);
}
```

```java
System.out.println(even( n: 1));
System.out.println(even( n: 2));
System.out.println(odd( n: 3));
System.out.println(odd( n: 4));
```

```
false
true
true
false
```

Verschachtelte Rekursion

- Argumente bei einem rekursiven Aufruf sind selbst wieder Rekursionsaufrufe
- Vorhergehender Aufruf muss abgeschlossen sein, bevor ein weiterer gestartet werden kann
- Beispiel: Moduloberechnung

```
private int modulo(int a, int b) {
    if(a < b) {
        return a;
    } else if(a < 2 * b) {
        return a - b;
    } else {
        return modulo(modulo(a, b: 2 * b), b);
    }
}
```

```
System.out.println(modulo( a: 3,  b: 5));
System.out.println(modulo( a: 14,  b: 4));
System.out.println(modulo( a: 426374,  b: 485));
```

```
3
2
59
```

#3 Rekursion vs. Iteration

Vergleich



Rekursion und Iteration

- Rekursion und Iteration sind gleich mächtig
- Rekursion lässt sich in Iteration umformen und umgekehrt
  - Das sagt noch nichts über die Lesbarkeit, den Umfang und die Effizienz des Ergebnisses aus
- Entrekursivierung (Rekursion zu Iteration)
  - Einfach bei Endrekursion und linearer Rekursion
  - Meist umfangreicher bei verzweigter Rekursion
  - Umfangreicher bei verschachtelter Rekursion

Vergleich (1)

- Innerhalb eines Methodenrumpfs können mehrere rekursive Aufrufe erfolgen
  - Es ist aufwendig (aber möglich), solche rekursiven Methoden in Schleifen umzuformen
- Bestimmte Aufgaben lassen sich mit Rekursion kürzer formulieren

Vergleich (2)

- Bei der Rekursion können Daten weitergereicht werden
  - Durch Parameter
  - Durch Rückgabewerte, die wiederum in Berechnungen einfließen können
- In einer Schleife können Daten dagegen nur von der vorherigen zur nächsten Iteration fließen
  - Zur Kompensation sind häufig zusätzliche Variablen für Zwischenergebnisse notwendig
  - Komplexere Struktur durch zusätzliche Variablen

Vergleich (3)

- Stack-Speicher ist begrenzt
  - Reicht oft aus
  - Geht bei sehr tiefen Rekursionen (sehr viele Aufrufebenen, großes n) aus
- Schleifen gehen mit diesem Speicher weniger verschwenderisch um
  - Sind bei großem n vorteilhaft

Vergleich (4)

- Falsche oder fehlende Abbruchbedingung
  - Weder rekursive Methoden noch Schleifen terminieren auf normale Weise
- Rekursive Methoden brechen irgendwann ab
  - StackOverflowError
- Schleifen können ohne Fehlermeldung endlos laufen
  - Aufgrund der Fehlermeldungen sind solche Fehler bei Rekursionen leichter zu finden als bei Schleifen

Vergleich (5)

- Rekursion führt zu vielen kleinen Methoden, Schleifen eher zu größeren
- Bei Schleifen ist die Größe der Methoden besser steuerbar
- Rekursion kann sehr ineffizient sein (Bsp Fibonacci)